Assignment Beta - Question 2, Section 1

Student ID: 23k-6005
Topic: LoRA theory and numerical answers

(a) Number of trainable parameters in LoRA

Assume the original weight matrix is:

$$W \in \mathbb{R}^{4096 \times 4096}$$

In LoRA, the original weight matrix is frozen. Instead of training the full matrix, LoRA learns a low-rank update:

$$\Delta W = BA$$

where:

$$A \in \mathbb{R}^{16 \times 4096}$$

$$B \in \mathbb{R}^{4096 \times 16}$$

The LoRA rank is $r = 16$.

The number of trainable parameters is:

$$\text{Parameters in } A = 16 \times 4096 = 65,536$$

$$\text{Parameters in } B = 4096 \times 16 = 65,536$$

$$\text{Total trainable LoRA parameters} = 65,536 + 65,536 = 131,072$$

Therefore, LoRA trains **131,072 parameters** for this weight matrix.

(b) Percentage of parameters trained

If the full 4096×4096 weight matrix were fine-tuned, the number of trainable parameters would be:

$$4096 \times 4096 = 16,777,216$$

LoRA trains only 131,072 parameters. Therefore:

$$\text{Percentage trained} = \frac{131,072}{16,777,216} \times 100$$

$$= 0.78125\%$$

Therefore, LoRA trains only **0.78125%** of the parameters that full fine-tuning would train for this matrix.

(c) Why is B initialized with zeros and A initialized randomly?

LoRA uses the update:

$$W' = W + BA$$

At the start, B is initialized with zeros. Therefore:

$$BA = 0$$

So, at the beginning:

$$W' = W$$

This means the original pre-trained model behaves exactly the same before LoRA training starts. It gives a stable starting point.

A is initialized randomly so that the two matrices do not stay identical or produce the same updates. During training, gradients first update B. After B becomes non-zero, gradients can also update A. Together, they learn a useful small update for the original frozen weight matrix.

(d) Difference between LoRA, QLoRA, DoRA, and AdaLoRA

LoRA

LoRA freezes the original model weights and trains two small low-rank matrices, A and B. This reduces the number of trainable parameters and memory needed for fine-tuning.

QLoRA

QLoRA applies LoRA while the main model is loaded in low-bit quantized form, commonly 4-bit NF4. The base model needs less memory, while the LoRA adapters are trained separately. QLoRA also uses paged optimizers to help manage memory during training.

DoRA

DoRA separates a weight update into magnitude and direction. Instead of only learning a low-rank update, it handles how large the weight is and which direction it points. This can

give more flexible updates than standard LoRA.

AdaLoRA

AdaLoRA changes the rank allocation during training. More rank is given to important layers or matrices, while less important ones receive less rank. This uses the parameter budget more efficiently than using the same fixed rank everywhere.

Summary

For a 4096×4096 weight matrix with LoRA rank 16, LoRA trains **131,072 parameters**, which is only **0.78125%** of the parameters used by full fine-tuning. LoRA saves memory and training cost while keeping the original model weights frozen.